

UserReady Internship Project

AI-Driven Multi-Agent Log Analysis System

Built Using Lyzr & Google ADK (Python)

Contributors:

Dugini Sai Sharan | duginisaisharan@gmail.com

Kartik Bhada | kartikbhabawork@gmail.com





AI-Powered Multi-Agent Log Analysis System

Introduction



Modern software systems generate huge volumes of logs every second. When a crash occurs, developers must manually dig through thousands of lines to understand what went wrong, which is slow, repetitive, and often delays recovery. We are building an AI-driven log analysis system that continuously reads and analyzes logs, detects anomalies, identifies root causes, and provides clear, actionable insights instantly. This significantly reduces the time and effort required for troubleshooting and helps maintain system stability.

Our Solution

Our solution acts as a 24/7 intelligent log investigator. Instead of developers spending hours searching for when a crash started, what triggered it, or which component failed, the system summarizes everything in seconds. It also identifies recurring failures that often hide critical bugs, helping teams spot issues long before they become major incidents. This makes debugging faster, reduces downtime, and improves overall reliability.

Real-World Examples

- **Cloudflare Outage:** A config file grew unexpectedly and crashed core service, taking 1/5th of the internet down. Our system would have instantly flagged the spike and pinpointed the faulty file.
- **AWS DNS Failure:** Continuous DNS errors disrupted global services. Our system would have caught the repeating failures early and alerted engineers immediately.

a

CLOUDFLARE

Impact & Benefits

Our system transforms log analysis into a fast, automated, and highly accurate process. By filtering noise, grouping repeated errors, and pinpointing root causes instantly, it dramatically reduces the time developers spend troubleshooting. This leads to faster issue resolution, lower operational overhead, and a more stable application environment overall. With optimized log processing and minimal LLM usage, it delivers clear insights efficiently while keeping costs low enabling teams to maintain reliability and respond to problems with confidence.

Design & Implementation

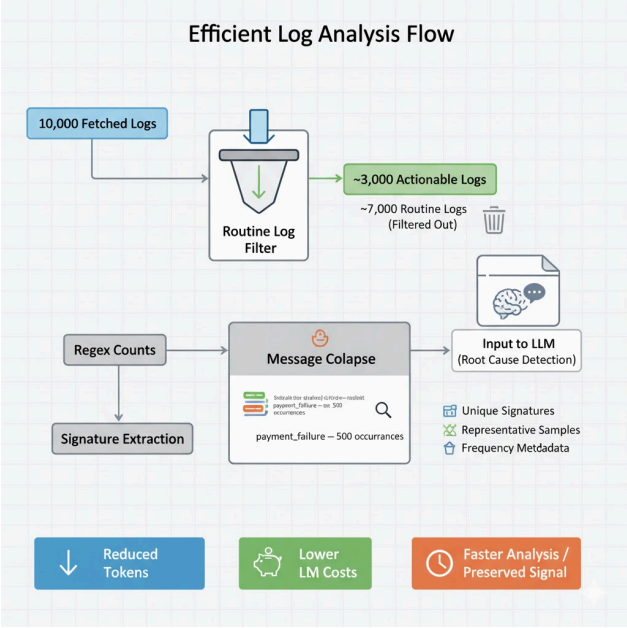
Technology & Overview

We built a production-ready, multi-agent log analysis system using **Google ADK In Python** (Agent Development Kit) as the orchestration layer and **AWS CloudWatch** *(a monitoring service for AWS cloud resources and applications that collects and tracks metrics, monitors log files, and sets alarms to react to changes)* as the log source. The system is designed to run on real-world workloads and prioritize cost-efficient LLM usage while delivering fast, actionable root-cause insights to engineering teams.

How the System Works (Simple Explanation)

Our system doesn't process all logs blindly. Instead, it works like a small AI team where each part has a simple job. It collects logs, removes noise, groups repeated errors, and gives the AI only the important information so it can explain the issue clearly.'

Smart Log Optimization (the 30% rule) Log Optimization (the 30% rule)

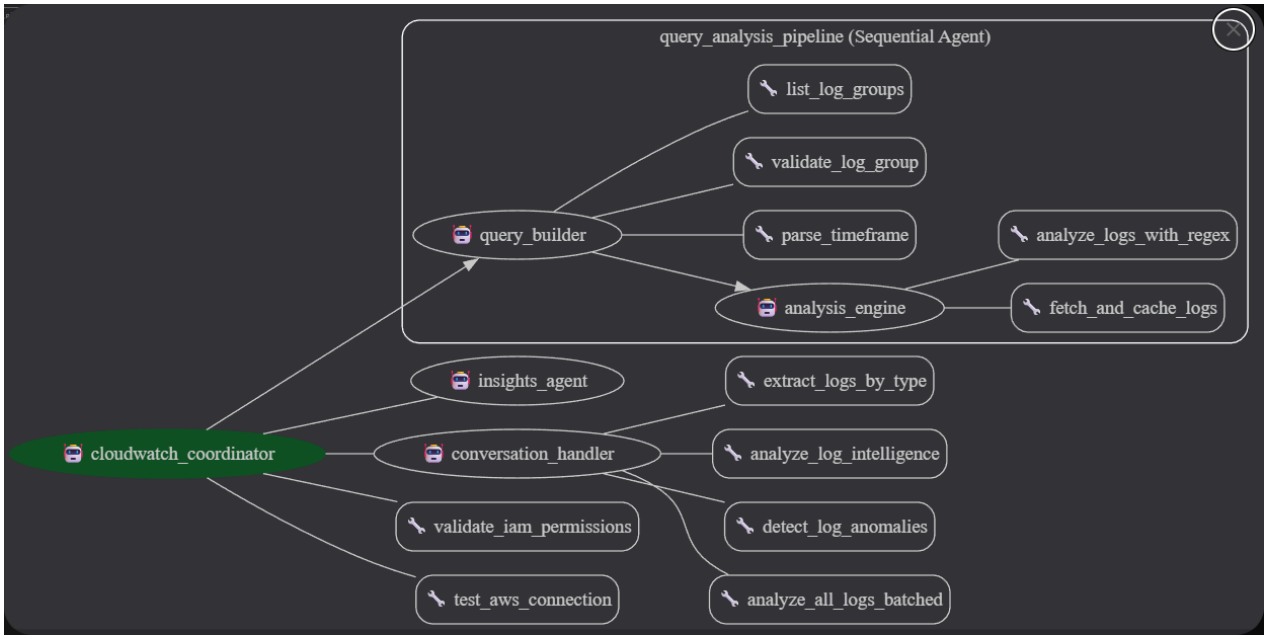


In real traffic, most log volume is routine noise. We implemented a contextual optimization:

- Example: from 10,000 fetched logs, ~7,000 are routine (success/heartbeat) and are **filtered out**.
- The remaining ~3,000 logs (errors/warnings/interesting events) are analyzed further via regex counts and signature extraction.
- Repetitive messages are **collapsed**: e.g., a payment-failure message that appears 500 times becomes a single signature with a frequency ("payment_failure – 500 occurrences").
- The LLM is given only the **unique signatures + representative samples + frequency metadata**, not all duplicates.

This reduces tokens dramatically, lowers LLM costs, and speeds up analysis while preserving signal needed for root-cause detection.

Key Agents (simple breakdown)



- **Query Builder Agent :** This agent parses user intent and builds validated CloudWatch query plans (timeframe, templates, regions).
- **Analysis EngineAgent :** This agent fetches logs, caches them, runs regex statistical analysis, and returns the LogSummary (counts & samples only).
- **Insights / Conversation Handler Agent:** This agent formats human-friendly summaries and runs deep analysis step LLM analysis on a tightly filtered subset when requested.

Benefits (short)

- **Faster MTTR** (minutes vs hours) : developers get clear root-cause hints quickly.
- **Lower LLM cost & predictable token usage** through filtering and signature collapsing.
- **Scales to real workloads** by offloading heavy counting to efficient Python regex/statistics and reserving LLM for high-value analysis.

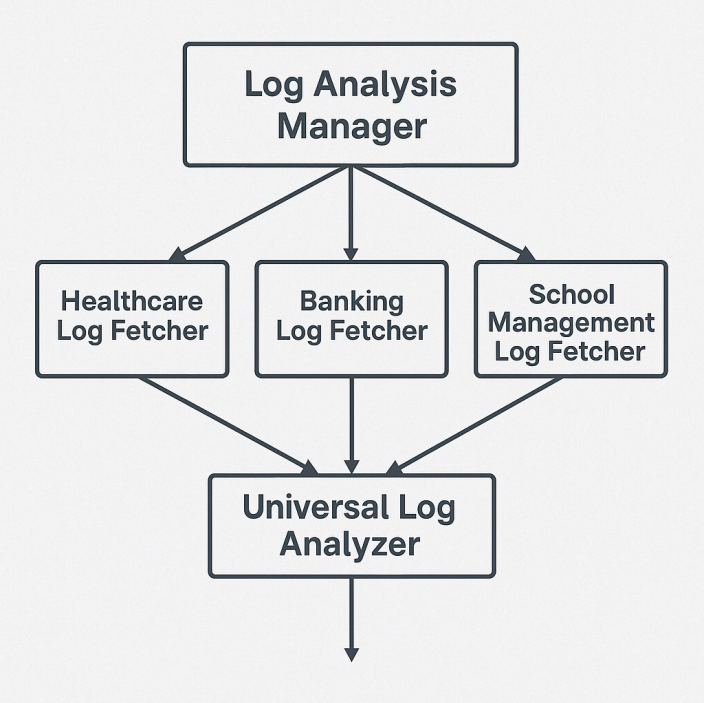
Lyzr-Based Log Analysis System

What We First Built (Lyzr Version)

Before moving to Google ADK, we initially built our log analysis system using **Lyzr**, where the entire workflow was powered by a simple multi-agent setup. The idea was the same: read logs, analyze them, and provide clear insights. But the structure and limitations were different.

How Our Lyzr Multi-Agent System Worked

We designed a very straightforward agent hierarchy:



1. Log Analysis Manager (Main Agent)

This was the central controller. Every user query first went to this agent. Based on the query (healthcare logs, banking logs, school logs, etc.), it decided which log-fetching agent should handle the request.

2. Log Fetcher Agents (3 Specialized Agents)

Each fetcher agent was responsible for one domain:

- **Healthcare Log Fetcher**
- **Banking Log Fetcher**
- **School Management Log Fetcher**

These agents fetched log data directly from their datasets (CSV files). Depending on the user instruction like *"show healthcare errors"* the relevant fetcher agent pulled only those logs.

3. Universal Log Analyzer Agent

After the fetchers collected logs, everything was passed to this single analyzer agent. This agent:

- Read the logs
- Tried to identify errors or unusual patterns
- Generated a simple explanation for the user

This gave us a functioning end-to-end log analysis flow using Lyzr.

Simple Lyzr Workflow

User → **Log Analysis Manager → Fetcher Agent (Healthcare/Banking/School) → Universal Log Analyzer →** Response

Why This Approach Was Limited & Why We Moved to Google ADK

The Lyzr version worked but had important limitations: no log optimization, no regex filtering, no pattern detection, and a very small context window . which caused missed errors and incomplete analysis. It also wasn’t suitable for large‑scale or real‑world log volumes.

Because of these gaps, we moved to **Google ADK**, which gives us full flexibility: Python‑based log optimization, regex and signature extraction, scalable multi‑agent workflow, 70–90% token reduction, and support for analyzing real CloudWatch logs.